

Section 2, Question 2

Real-Time Log Analytics with gRPC

Distributed Systems — Homework 3

1 What the question asks

Question 1 computed the Homework 2 Question 7 statistics over a file, as a batch job that reads everything and then finishes. This question asks for the same statistics over a **live stream**: records arrive continuously, and the answers must be available *while data is still arriving*.

The system must be distributed over several machines and built on gRPC, with four participants — a stream client that sends records, a coordinator that receives and distributes them, workers that count, and query clients that ask for the analytics at any moment. The final answer must still equal the sequential program's, byte for byte.

2 Architecture

RPC	gRPC shape	Purpose
<code>StreamRecords</code>	client-streaming	the entire log stream, as one call
<code>GetAnalytics</code>	unary	one question, one answer
<code>WatchAnalytics</code>	server-streaming	the dashboard is pushed updates
<code>Worker.Process</code>	client-streaming (internal)	coordinator → worker
<code>Worker.GetStats</code>	unary (internal)	coordinator asks for totals

All four gRPC call shapes are used, each where it genuinely fits rather than for the sake of coverage. On the cluster the processes are placed one per machine: the coordinator on the first, the clients on the second, one worker on each remaining machine, so nothing shares a CPU.

2.1 Design decisions, and why

The whole stream is one RPC. `StreamRecords` is a single client-streaming call rather than one call per batch, so the cost of establishing a call is paid once. gRPC's own flow control then provides backpressure without any code: if the coordinator cannot keep up, sending simply slows down at the client.

Records travel in columnar batches. A `RecordBatch` holds one list per field — all the timestamps, then all the server identifiers — rather than a list of record objects. Protobuf encodes a list of integers far more compactly than the same integers scattered across many small messages. This turns out to be the single largest performance lever in the system.

Workers send totals, not deltas. A worker always reports its full running totals, and the coordinator *replaces* that worker's contribution rather than adding to it. This makes asking safe to repeat: if a reply is lost, the coordinator simply asks again, and nothing is counted twice. An earlier design used epoch-numbered deltas with acknowledgements; it was the hardest part of the system to explain, and removing it lost nothing.

Bounded queues. Each worker has a fixed-size queue. When a worker falls behind, its queue fills, the coordinator blocks, and that blocking propagates back through gRPC to the client. Memory stays bounded rather than growing until the process dies.

Mergeability and integer time. As in Question 1, every figure is a count, sum, minimum or maximum, so partial results combine in any order. Averages are never stored — a sum and a count are, and the division happens at printing time. Response times are whole numbers of millionths of a millisecond, so no ordering of workers can change the final digit.

3 Results

One million records, four workers unless stated, median of three runs, every run checked against the sequential program.

3.1 Workers

Workers	Throughput	Speedup
1	823,037 rec/s	1.00×
2	1,573,037 rec/s	1.91×
4	2,909,799 rec/s	3.54×

Scaling is close to linear because workers never communicate with one another: each holds its own totals and is merged only when someone asks. The shortfall from 4× is the coordinator, which parses and routes every record itself and is therefore the one part that cannot be divided.

3.2 Message granularity — the dominant factor

Records per message	Throughput	vs one at a time
1	10,211 rec/s	1×
10	99,199 rec/s	10×
100	832,628 rec/s	82×
1,000	2,909,799 rec/s	285×
10,000	3,124,137 rec/s	306×

This spans more than two orders of magnitude and dwarfs every other variable measured. Each message carries a fixed cost — a protobuf header, a length prefix, a pass through gRPC’s machinery, a queue operation — and at one record per message that fixed cost *is* the workload. The curve flattens beyond a thousand records per message, where the fixed cost has become negligible relative to the payload, so a thousand is the default.

The practical lesson generalises beyond this system: when an RPC framework seems slow, the first thing to examine is how much work each call carries, not the framework.

3.3 Routing strategy

Strategy	Throughput	How it chooses
<code>round_robin</code>	2,909,799 rec/s	next worker in turn
<code>least_loaded</code>	1,822,886 rec/s	the worker with the shortest queue
<code>hash_server</code>	221,908 rec/s	by server identifier

Even splitting beats clever splitting, and by a wide margin. `hash_server` sends every record for a given server to the same worker; the dataset contains a handful of busy servers, so one worker receives most of the traffic while the others idle — $13\times$ slower. `least_loaded` inspects queue depths on every batch, and that inspection costs more than the imbalance it avoids.

Since the analytics are mergeable, there is no correctness reason to keep a server’s records together, so the simplest possible policy is also the fastest.

3.4 The cost of querying during ingestion

Query clients	Ingest throughput	Query p99
0	2,909,799 rec/s	—
1	2,388,961 rec/s	32 ms
4	779,990 rec/s	42 ms
16	197,263 rec/s	192 ms

Queries *are* served while data streams in, which is what the question asks for — but they are not free. Sixteen closed-loop query clients cost $15\times$ the ingestion rate. Each query makes the coordinator collect and merge every worker’s totals, and that work competes directly with routing records.

This is precisely why `WatchAnalytics` exists. A dashboard subscribed to the server stream is *pushed* updates from one periodic merge, so ten observers cost one round of work rather than ten. Polling with `GetAnalytics` is the expensive pattern; subscribing is the cheap one.

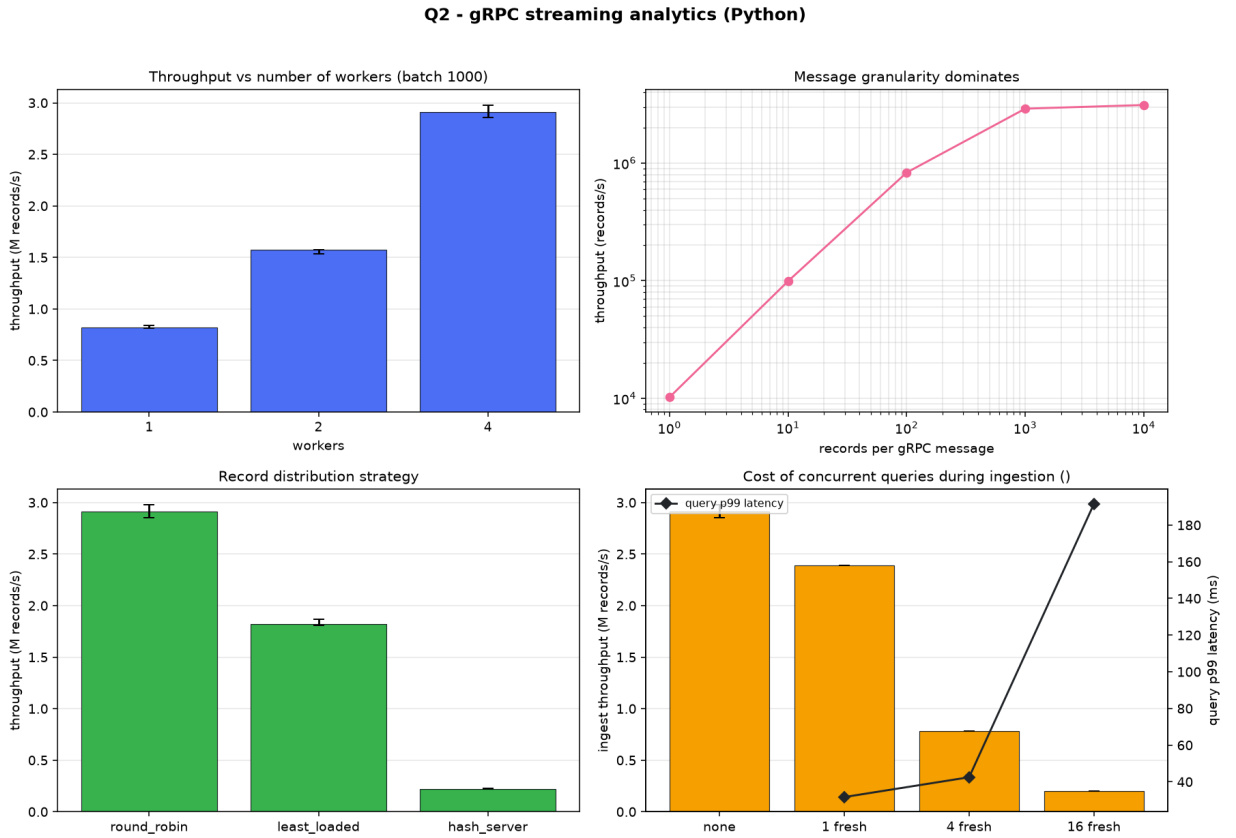


Figure 1: The four sweeps: throughput against worker count; the effect of message granularity, spanning two orders of magnitude; the comparison of routing strategies; and the cost of concurrent queries during ingestion, with query latency on the right-hand axis.

Q2 gRPC - the four claims, and the measurements behind them

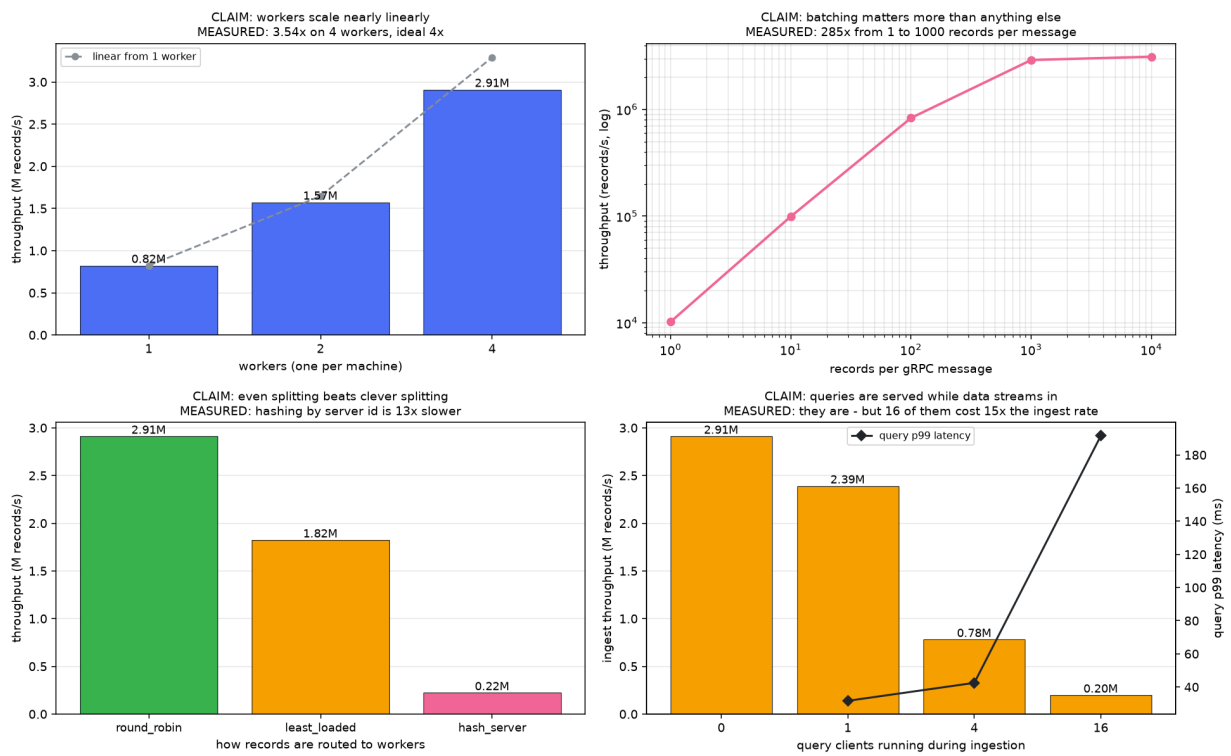


Figure 2: Each claim made about the system, set beside the measurement that supports it. Stating a claim and its evidence together is what distinguishes a design that was measured from one that was merely asserted.

4 Comparison with the batch implementations

All three implementations compute the same analytics over the same one-million-record file on four machines.

Implementation	Throughput	Time for 1M records
MapReduce (Question 1)	1.35M rec/s	0.74 s
MPI (Homework 2 baseline)	2.09M rec/s	0.48 s
gRPC streaming (this work)	2.91M rec/s	0.34 s

This ordering is **not** a ranking of the three technologies, and it would be dishonest to present it as one. MapReduce and MPI read the file from disk and terminate; this system receives records over the network and continues answering queries throughout, and its figure excludes reading a file because a real stream has no file to read.

What the comparison does establish is worth stating plainly: the streaming design pays **no penalty for being incremental**. Maintaining a queryable answer at every instant — rather than producing one answer at the end — costs nothing in throughput here. The batch systems have no advantage to give up.

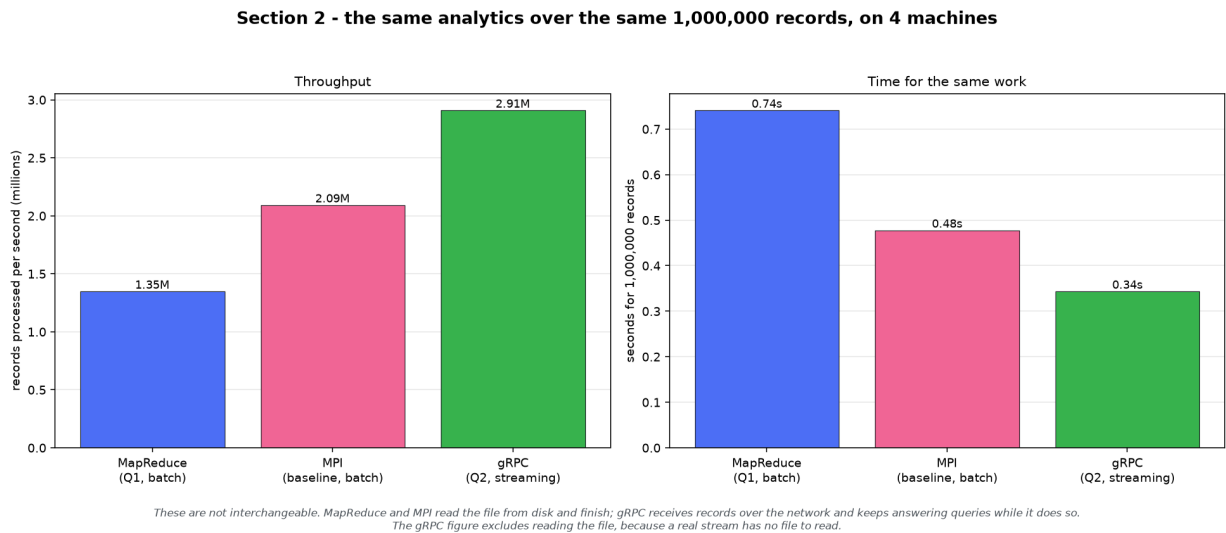


Figure 3: MapReduce, MPI and gRPC over identical work on four machines. The caveat about what is and is not being compared is printed on the figure itself, so it cannot become separated from the numbers.

5 Correctness

Seventy-six checks, all passing. They sweep worker counts, all three routing strategies, batch sizes from 1 to 10,000, several concurrent stream sources, and queries issued mid-stream — each time comparing the system’s final answer against Question 1’s `log_seq`, the unmodified Homework 2 sequential program. Using one oracle for both questions rather than two means the two implementations cannot quietly disagree with each other.

The checks that matter most are the ones where the answer could plausibly have differed: several sources streaming at once, a query issued halfway through, and a reset followed by a fresh stream. Distribution is only correct if the result does not depend on timing, and those are the cases where timing varies.

6 Conclusions

The system sustains roughly 2.9 million records per second on four machines while answering queries concurrently, and its final answer is byte-identical to the sequential program under every configuration tested.

Three findings are worth carrying forward. Message granularity dominates everything else, by $285\times$ — far more than worker count or routing policy. The simplest routing policy is the fastest, because mergeable analytics impose no requirement to keep related records together. And concurrent queries are genuinely affordable only when observers subscribe rather than poll, which is what the server-streaming RPC is for.